

Side — AI Microservices Developer: Model Answers (Q3–Q43)

These are drafted starting points based on your real projects. Read them, then say them in your own words — don't memorize verbatim, interviewers can tell.

Background & Fit

Q3. What does “AI Microservices Developer” mean to you, and how does it differ from a generic ML engineer role? To me it's less about training models and more about *productionizing* intelligence — wrapping LLMs and generative models behind clean, reliable APIs that other systems and teams can call. A generic ML engineer role often centers on model development, feature engineering, training loops. This role is closer to backend engineering with an AI core: API design, uptime, latency, scalability, and translating fuzzy business asks into deterministic service contracts — which is exactly what I've been doing with Bedrock/Claude-backed FastAPI services.

Q4. Tell me about a time you had to manage a project independently, end-to-end. Use the LangGraph IT ticketing system: you likely owned scoping (why a linear chain failed), design (StateGraph, router, sub-agents), build, and handling edge cases (confidence-based fallback). Structure as: *situation* → *what you owned* → *key decisions* → *outcome*. Emphasize you gathered requirements from stakeholders, not just wrote code.

Python / API / Microservices

Q5. Walk me through the architecture of a Python microservice you built from scratch. Pick the Bedrock Agent Assist service. Describe: FastAPI entrypoint → request validation → orchestration layer (routing to Bedrock/Claude or LangGraph flow) → downstream calls (Kendra/OpenSearch retrieval, enterprise APIs) → response shaping → deployment on ECS with DevOps. Mention async handling for LLM latency and how you kept the service stateless/horizontally scalable.

Q6. FastAPI vs Django vs Flask — when would you choose each? FastAPI: async-first, auto OpenAPI docs, ideal for LLM-calling services where I/O-bound latency matters and I need Pydantic validation for structured LLM outputs (ties directly to your invoice pipeline

work). Django: when I need batteries-included — ORM, admin, auth — for a broader web app, less ideal for lightweight microservices. Flask: minimal, good for small internal tools, but I'd reach for FastAPI now given async + typing benefits for AI workloads.

Q7. How do you handle authentication/authorization across microservices? Talk about API keys/OAuth2/JWT between services, service-to-service auth (IAM roles in AWS context), and how enterprise API calls (like the password-reset tool in your ticketing agent) were authenticated. Mention least-privilege access per microservice.

Q8. How do you design an API that needs to call an LLM but stay responsive under load? Async request handling (FastAPI's async def + httpx/boto3 async clients), streaming responses where possible, setting timeouts with graceful degradation, queuing/background workers for longer-running LLM calls, and circuit breakers if the LLM provider is slow/down.

Q9. How would you version an API that's evolving alongside changing LLM prompts/models? URL or header-based API versioning (/v1/, /v2/), decoupling prompt templates from application code (config-driven, versioned separately), and running shadow/canary tests when swapping model versions before full rollout.

Q10. How do you handle rate limits and retries when calling a third-party LLM API? Exponential backoff with jitter, respecting provider-specific rate-limit headers, circuit breaker pattern to stop hammering a failing provider, and fallback to a secondary model/provider if primary is unavailable — relevant given this role wants multi-provider flexibility.

Q11. Describe your approach to logging, monitoring, and debugging a production microservice. Structured logging (JSON logs) with request IDs/trace IDs for cross-service tracing, CloudWatch/equivalent dashboards, alerting on error rate and latency percentiles (p95/p99), and specifically for LLM calls — logging prompt/response pairs (redacted for PII) to debug quality issues like the hallucination problem you fixed for the legal client.

Q12. How do you structure a microservice to be provider-agnostic (swap OpenAI ↔ Claude ↔ open-weights)? Abstract the LLM call behind an interface/adaptor layer (e.g., a `LLMClient` base class with provider-specific implementations), keep prompts and parsing logic provider-agnostic where possible, and centralize provider selection via config/env rather than hardcoding — this is a good place to mention LangChain's abstraction layer, which you already have hands-on experience with.

LangGraph / Multi-Agent (IT ticketing project)

Q13. Walk me through the StateGraph design for your IT support ticketing system. Nodes: Router Agent (classifies intent) → conditional edges to RAG Agent (KB questions) or

Tool-Executing Agent (actions like password reset) → shared state object carrying conversation history, retrieved context, and confidence scores → fallback edge to human handoff.

Q14. How did the Router Agent decide which path to take? What if it misclassified?

Likely an LLM classification call against defined intent categories, possibly with few-shot examples. For misclassification: describe any confidence threshold check, and how a wrong route was recoverable (e.g., RAG Agent could hand back to Router if it found no relevant docs, rather than dead-ending).

Q15. How is state persisted/passed between agents in LangGraph? LangGraph's shared state object (TypedDict/Pydantic model) flows through nodes; each node reads/updates relevant fields (conversation history, retrieved docs, confidence score). Mention if you used LangGraph's checkpointing for persistence across sessions.

Q16. What happens when the LLM needs clarification — how did you implement the loop without infinite retries? A clarification node loops back to the user, incrementing a counter in state; after N attempts (e.g., 2–3) without resolution, force fallback to human agent rather than looping indefinitely — good place to mention explicit loop guards as a production-safety detail.

Q17. How did you compute or calibrate the confidence score that triggered human fallback? Could be model-reported confidence/logprobs, a secondary classifier, or a self-critique step similar to your Self-RAG loop. Be honest about how the 80% threshold was chosen — likely empirically tuned against a labeled test set.

Q18. What were the failure modes of this system, and how did you catch them?

Misrouted intents, retrieval returning irrelevant KB articles, tool calls failing against enterprise APIs (auth/timeout issues), infinite clarification loops before the guard was added. Caught via logging + human-in-the-loop review of low-confidence cases.

Q19. How would you test a multi-agent LangGraph flow? Unit tests per node (mock LLM responses), integration tests for full graph traversal with golden test cases per intent category, and regression tests when prompts/models change.

Q20. How does LangGraph compare to just chaining function calls yourself?

LangGraph gives you explicit state management, conditional branching, cycles/loops (which plain chains can't do easily), and visualization/debuggability of the flow — versus hand-rolled orchestration where state passing and loop control become messy fast, especially with multiple agents.

Q21. What was causing the hallucinations in the original pipeline before your fix? Likely naive single-pass retrieval + generation with no verification step — vague queries retrieving poor matches, and no check that the generated answer actually grounded in retrieved docs.

Q22. Explain query transformation — what made a query “vague,” and how did you rewrite it? Short/ambiguous user queries lacking context (e.g., “what’s the penalty?”) were expanded/rewritten using an LLM step to add specificity before retrieval — improving match quality against the vector store.

Q23. Why a Parent Document Retriever specifically — what problem does it solve over standard chunking? Standard chunking retrieves small, isolated chunks that can lack surrounding context. Parent Document Retriever indexes small chunks for precise matching but returns the larger parent document/section for generation — balancing retrieval precision with contextual completeness, important for legal/compliance text where context matters.

Q24. Walk me through the Self-RAG critique loop — what did the critique prompt actually check? A second LLM call compared the generated answer against the retrieved source documents, checking factual alignment/grounding. If misaligned, the chain reformulated the prompt (e.g., forcing stricter citation) and retried generation.

Q25. What happens if the critique loop keeps failing — is there a retry limit? Yes — describe a max retry count (e.g., 2) before falling back to a safe response like “insufficient grounded information found” rather than looping indefinitely or returning a low-confidence answer.

Q26. How did you measure whether hallucination rate actually improved? Before/after comparison on a labeled eval set — manual review or an automated grounding-check metric (e.g., % of claims traceable to source docs), tracked over time.

Q27. What vector database did you use and why? If OpenSearch (consistent with your other work) — mention it as AWS-native, supports hybrid search (keyword + vector), and integrates well with Bedrock in your existing stack.

Q28. How do you evaluate RAG quality in general? Retrieval metrics (precision/recall @k), generation metrics (faithfulness/groundedness, answer relevance), and human eval spot-checks — frameworks like RAGAS are worth name-dropping if you’ve looked at them.

Structured Output / Invoice Pipeline

Q29. Walk me through the PydanticOutputParser setup — how did you define the schema? Defined a Pydantic model matching expected invoice fields (vendor, line items,

dates, totals) with type constraints; LangChain's parser injects format instructions into the prompt and validates/parses the LLM's JSON output against that schema.

Q30. What kinds of formatting failures did you see, and how did the self-correction loop work? Malformed JSON, missing required fields, wrong types (e.g., date as string vs. expected format). On validation failure, the Pydantic error message was fed back into a retry prompt telling the LLM exactly what was wrong, so it could self-correct.

Q31. How do you handle partially-correct structured outputs? Field-level validation — accept valid fields, flag/re-prompt only for failing fields rather than discarding the whole response; log partial failures for human review if retries are exhausted.

Q32. How would you scale this pipeline to handle thousands of documents per day? Move from synchronous processing to a queue-based architecture (SQS + Lambda/worker pool), batch processing where possible, parallelize independent document extractions, and monitor cost/throughput tradeoffs of LLM calls at scale.

LLM/GenAI General

Q33. How do you keep up with new LLM releases, and how would you evaluate a new model for a use case here? Follow provider release notes/benchmarks, but more importantly — run task-specific eval sets against your own use cases rather than trusting general benchmarks, since a model that's great at coding may not be best at structured extraction or grounded RAG.

Q34. Open-weights vs hosted APIs — how do you decide? Hosted APIs (OpenAI/Claude): faster to integrate, no infra to manage, better for rapidly evolving use cases. Open-weights (Llama, Mistral): better for cost control at scale, data residency/privacy needs, or fine-tuning for a narrow domain task — you have direct fine-tuning experience with Mistral/Phi to point to here.

Q35. Approach to prompt versioning and testing changes without breaking production? Store prompts as versioned config (not hardcoded), run new prompt versions against a regression test set before deploying, and consider canary/A-B rollout for high-traffic flows.

Q36. Approach to cost control when a service makes many LLM calls? Caching repeated queries, using smaller/cheaper models for simple sub-tasks (routing) and reserving larger models for complex generation, batching where possible, and monitoring token usage per endpoint to catch runaway costs.

Q37. How would you design an AI image generation pipeline? Be honest this is a gap

area — frame it as: API wrapping a diffusion model (Stable Diffusion/DALL-E-style), async job queue since generation is slow, S3 storage for outputs, webhook/polling for completion status. Say you'd ramp quickly given your existing async/microservice patterns even without direct image-gen experience yet.

System Design

Q38. Design an API that lets any team in the company submit a business process for LLM-based automation. A generic "automation request" service: intake API defining process steps/rules, an LLM-driven interpreter that maps the request to available tool/agent capabilities (similar to your Router Agent pattern), approval/review step before going live, and a monitoring dashboard per automated process.

Q39. How would you architect a system that needs to support multiple LLM providers interchangeably? Same adapter/interface pattern as Q12 — provider-agnostic request/response contracts, config-driven provider selection, fallback chains if a provider is down, and centralized cost/usage tracking across providers.

Q40. How do you handle sensitive data (PII) in requests going to third-party LLM APIs? PII detection/redaction before sending to external APIs (Comprehend-style entity detection, which you have experience with), data residency awareness per provider's data handling policy, and audit logging of what was sent where.

Behavioral

Q41. Tell me about a time your AI/LLM solution didn't work as expected — what did you do? Good candidate: the original hallucination problem on the legal client project, or an early version of the ticketing router misclassifying intents. Structure as STAR: what broke, how you diagnosed it, the fix (Self-RAG loop / confidence threshold), and the measurable outcome.

Q42. Describe a disagreement with a stakeholder about requirements and how you resolved it. Have a real, specific example ready — even a small one (e.g., a business team wanting a shortcut that would've hurt accuracy). Focus on how you presented tradeoffs with data, not just "I convinced them."

Q43. How do you prioritize when a business team wants a quick prototype vs. production-grade quality? Talk about scoping a fast PoC to validate the approach/value first, being explicit about what's cut (error handling, scale, security) in a prototype, then defining a clear path to production-hardening once validated — avoids over-engineering

something that might get killed, while not shipping something fragile as “done.”

Tip: for every technical answer above, be ready for a natural follow-up “why did you choose X over Y” — that’s usually where interviewers probe whether you actually built it or are reciting a description.